

PROYECTO INTEGRADOR

Plataforma de Administración y Control de un Curso Universitario

Equipo: Artemis 3

Integrantes (Nombre - Padron):

Julieta Paola, Fernández - 115660

Celeste Abigail, Funes Taya - 115585

Joaquín, Tapia Castañeda - 114342

Nicolás, Pattini - 114947

Iván Nahuel, Avillo Llanos - 112713

Lucas, Calahuana - 114651

Lucca, Moyano - 113987

Luna Abril, Alonso Díaz - 115439

David, García - 115695

Tutor:

Tommy Villegas

ÍNDICE

1. HIPÓTESIS Y SUPUESTOS.....	3
Contexto del Proyecto.....	3
Supuestos Iniciales:.....	3
2. ALCANCE DEL PROYECTO.....	4
2.1 - INCLUYE (Requisitos Obligatorios).....	4
2.2 - NO INCLUYE EN ESTA FASE.....	4
3. BACKLOG PRIORIZADO - 4 FASES.....	5
FASE 1: Setup e Infraestructura (13-20 de mayo).....	5
FASE 2: ABMs y Autenticación (20-27 de mayo).....	6
FASE 3: QR y Reportes (27 mayo - 3 junio).....	7
FASE 4: Polish e Integración (3-8 de junio).....	8
4. DETALLE DE TAREAS POR TIPO DE DESARROLLO.....	9
4.1 - BACKEND (FLASK).....	9
Endpoints a Desarrollar:.....	9
4.2 - FRONTEND (JINJA2 + HTML/CSS/JAVASCRIPT).....	10
Páginas Principales:.....	10
4.3 - BASE DE DATOS (MYSQL).....	11
Tablas Principales:.....	11
5. MOCKUPS DEL SISTEMA.....	12
5.1 - Pantalla de Inicio y Login.....	12
5.2 - Dashboard del Alumno.....	12
5.3 - Dashboard del Profesor.....	12
6. MODELO DE BASE DE DATOS.....	13
6.1 - Tablas Principales:.....	13
6.2 - Relaciones Principales.....	13
7. PRÓXIMOS PASOS.....	14

1. HIPÓTESIS Y SUPUESTOS

Contexto del Proyecto

La plataforma será utilizada para gestionar de manera integral un curso universitario, permitiendo a profesores administrar alumnos, evaluaciones, notas y asistencia, mientras que los alumnos pueden consultar sus calificaciones y participar en el sistema de asistencia.

Supuestos Iniciales:

- 1 curso universitario único en la plataforma
- Entre 10 a 13 profesores como máximo con rol de administrador/profesor
- Aproximadamente 200 alumnos registrados
- Los usuarios accederán desde navegadores web modernos
- La conexión a internet estará disponible para todos los usuarios
- Profesores cargarán notas y asistencia de forma manual
- El QR dinámico será enviado por email al inicio de cada clase
- Base de datos MySQL disponible y accesible
- Acceso al sistema durante 24 horas
- Uso principalmente desde dispositivos de escritorio y tablets

2. ALCANCE DEL PROYECTO

2.1 - INCLUYE (Requisitos Obligatorios)

Sistema de login y registro diferenciado para profesores y alumnos

ABM completo de alumnos con importación desde archivo CSV

ABM de evaluaciones (parciales, parcialitos, trabajos prácticos)

ABM de equipos asociados a trabajos prácticos

Carga y gestión de calificaciones por tipo de evaluación

Sistema de asistencia mediante QR dinámico (enviado por email)

Informes en PDF: Listado de alumnos, Estadísticas de aprobación, Listado de equipos

Log de actividades y registro de acceso de usuarios

Seguridad en todos los endpoints (autenticación y autorización)

Perfil de usuario con datos personales

2.2 - NO INCLUYE EN ESTA FASE

Aplicación móvil con Kivy UI

Material de estudio público (sin autenticación)

Dockerización de la aplicación

Hosting en PythonAnywhere u otra plataforma

Integración con sistemas externos

Aplicación de escritorio

3. BACKLOG PRIORIZADO - 4 FASES

FASE 1: Setup e Infraestructura (13-20 de mayo)

Objetivo: Sentar las bases del proyecto, estructura y conexión a base de datos

1) **Backend:**

- a) Estructura y carpetas del proyecto Flask
- b) Configurar conexión a MySQL y ORM (SQLAlchemy si aplica)
- c) Setup de autenticación (sesiones o JWT)
- d) Endpoint POST /api/login - Profesor
- e) Endpoint POST /api/login - Alumno
- f) Endpoint POST /api/register - Alumno
- g) Validación de tokens en endpoints
- h) Manejo de errores y excepciones

2) **Frontend:**

- a) Estructura base con Jinja2 + framework(si lo permite el tutor)
- b) Template principal (base.html)
- c) Página login profesor
- d) Página login alumno
- e) Página registro alumno
- f) Formularios con validación HTML5
- g) Navbar y menú de navegación

3) **Base de Datos:**

- a) Tabla usuarios (id, email, password, rol, created_at)
- b) Tabla profesores (id, nombre, email, departamento)
- c) Tabla alumnos (id, nombre, legajo, email, estado)
- d) Tabla cursos (id, nombre, año, semestre)
- e) Script create_tables.sql
- f) Script seed_data.sql con datos de prueba
- g) Índices en columnas de búsqueda frecuente

FASE 2: ABMs y Autenticación (20-27 de mayo)

Objetivo: Implementar ABMs completos y funcionalidad de importación CSV

1) **Backend:**

- a) CRUD completo alumnos
- b) Endpoint POST /api/alumnos - Crear
- c) Endpoint GET /api/alumnos - Listar
- d) Endpoint GET /api/alumnos/<id> - Obtener
- e) Endpoint PUT /api/alumnos/<id> - Actualizar
- f) Endpoint DELETE /api/alumnos/<id> - Eliminar
- g) Endpoint POST /api/alumnos/import-csv - Importación CSV
- h) ABM Evaluaciones
- i) ABM Equipos
- j) Validación de permisos por rol

2) **Frontend:**

- a) Dashboard profesor - Listado alumnos
- b) Dashboard alumno - Bienvenida
- c) Formulario ABM alumnos
- d) Interfaz importar CSV (drag and drop)
- e) Listado de alumnos (tabla)
- f) Formulario ABM evaluaciones
- g) Formulario ABM equipos
- h) Mensajes de error y validación visual

3) **Base de Datos:**

- a) Tabla evaluaciones (id, nombre, tipo, fecha, curso_id)
- b) Tabla equipos (id, nombre, trabajo_practico_id)
- c) Tabla equipo_alumnos (equipo_id, alumno_id)
- d) Tabla evaluacion_alumnos (evaluacion_id, alumno_id)
- e) Relaciones y foreign keys completas
- f) Datos de prueba adicionales

FASE 3: QR y Reportes (27 mayo - 3 junio)

Objetivo: Implementar sistema de asistencia con QR y generación de informes PDF

1) **Backend:**

- a) Generador QR dinámico (código único por clase/fecha)
- b) Endpoint POST /api/asistencia/generar-qr - Crear QR
- c) Endpoint POST /api/asistencia/registrar - Registrar asistencia
- d) Endpoint GET /api/asistencia - Consultar asistencias
- e) Generador de informes PDF - Listado alumnos
- f) Generador de informes PDF - Estadísticas aprobación
- g) Generador de informes PDF - Listado equipos
- h) Endpoint para descargar reportes
- i) Envío de QR por email

2) **Frontend:**

- a) Página asistencia (mostrar QR)
- b) Formulario registro asistencia
- c) Interfaz generador reportes
- d) Listado asistencias
- e) Botones descarga PDF
- f) Filtros para reportes
- g) Vista previa de reportes

3) **Base de Datos:**

- a) Tabla asistencia (id, alumno_id, fecha, qr_code, registrado_en)
- b) Tabla notas (id, alumno_id, evaluacion_id, calificacion, fecha)
- c) Tabla logs (id, usuario_id, accion, tabla_afectada, fecha)
- d) Vistas para reportes de estadísticas
- e) Índices para búsquedas de reportes

FASE 4: Polish e Integración (3-8 de junio)

Objetivo: Integración final, testing y refinamiento

1) **Backend:**

- a) Endpoint carga de notas (POST /api/notas)
- b) Endpoint consulta notas (GET /api/notas)
- c) Validación completa de datos
- d) Manejo de errores exhaustivo
- e) Testing de endpoints

2) **Frontend:**

- a) Dashboard profesor - Resumen completo
- b) Dashboard alumno - Notas y asistencia
- c) Perfil de usuario (alumno)
- d) Perfil de usuario (profesor)
- e) Calendario de eventos
- f) Testing de interfaz
- g) Optimización visual

3) **Base de Datos:**

- a) Optimización de queries
- b) Validación de integridad referencial
- c) Backup script
- d) Documentación de tablas

4. DETALLE DE TAREAS POR TIPO DE DESARROLLO

4.1 - BACKEND (FLASK)

El equipo de backend será responsable de desarrollar la lógica del servidor, endpoints RESTful, autenticación, conexión a base de datos y generación de reportes.

Endpoints a Desarrollar:

- POST /api/auth/login - Autenticación
- POST /api/auth/logout - Cerrar sesión
- POST /api/auth/register - Registro alumno
- GET/POST/PUT/DELETE /api/alumnos - CRUD alumnos
- POST /api/alumnos/import-csv - Importar desde CSV
- GET/POST/PUT/DELETE /api/evaluaciones - CRUD evaluaciones
- GET/POST/PUT/DELETE /api/equipos - CRUD equipos
- POST /api/notas - Cargar notas
- GET /api/notas - Consultar notas
- POST /api/asistencia/generar-qr - Generar QR
- POST /api/asistencia/registrar - Registrar asistencia
- GET /api/asistencia - Consultar asistencias
- GET /api/reportes/alumnos - Reporte alumnos PDF
- GET /api/reportes/estadisticas - Estadísticas PDF
- GET /api/reportes/equipos - Equipos PDF
- GET /api/profile - Perfil usuario

4.2 - FRONTEND (JINJA2 + HTML/CSS/JAVASCRIPT)

El equipo de frontend desarrollará las interfaces usando Jinja2 como motor de plantillas, HTML5, CSS con Bootstrap, y JavaScript vanilla (sin frameworks complejos al inicio).

Páginas Principales:

- index.html - Página de inicio
- login.html - Login profesor/alumno
- register.html - Registro alumno
- dashboard_profesor.html - Inicio profesor
- dashboard_alumno.html - Inicio alumno
- alumnos.html - Gestión alumnos
- evaluaciones.html - Gestión evaluaciones
- equipos.html - Gestión equipos
- notas.html - Carga de notas
- asistencia.html - Generador QR asistencia
- reportes.html - Descarga de reportes
- perfil.html - Perfil usuario
- calendario.html - Calendario de eventos

4.3 - BASE DE DATOS (MYSQL)

La base de datos será el corazón del sistema, almacenando todos los datos del curso, alumnos, evaluaciones, notas y asistencia. Se utilizará MySQL con un mínimo de 8 tablas.

Tablas Principales:

- usuarios - Almacena credenciales y roles
- profesores - Información de profesores
- alumnos - Información de alumnos
- cursos - Información del curso
- evaluaciones - Tipos de evaluación
- equipos - Equipos de trabajo
- equipo _alumnos - Relación equipos-alumnos
- notas - Calificaciones de alumnos
- asistencia - Registro de asistencia con QR
- logs - Historial de actividades

5. MOCKUPS DEL SISTEMA

5.1 - Pantalla de Inicio y Login

La pantalla de inicio mostrará dos opciones principales: login de profesor y login de alumno. Desde aquí también se podrá acceder a la sección de material público (deseable pero no obligatorio).

5.2 - Dashboard del Alumno

El alumno verá su perfil con:

notas por evaluación, asistencias registradas, equipos a los que pertenece y calendario de eventos/evaluaciones.

5.3 - Dashboard del Profesor

El profesor tendrá acceso a:

ABM de alumnos (crear, editar, eliminar, importar CSV), gestión de evaluaciones, carga de notas, generación de QR de asistencia, consulta de reportes y perfil.

6. MODELO DE BASE DE DATOS

6.1 - Tablas Principales:

- Tabla 'usuarios': (id, email, password, rol, created_at, updated_at)
- Tabla 'profesores': (id, usuario_id, nombre, departamento)
- Tabla 'alumnos': (id, usuario_id, nombre, legajo, estado, curso_id)
- Tabla 'cursos': (id, nombre, año, semestre, profesor_id)
- Tabla 'evaluaciones': (id, nombre, tipo, fecha, curso_id)
- Tabla 'equipos': (id, nombre, curso_id)
- Tabla 'equipo_alumnos': (equipo_id, alumno_id)
- Tabla 'notas': (id, alumno_id, evaluacion_id, calificacion, fecha)
- Tabla 'asistencia': (id, alumno_id, fecha, qr_code, registrado_en)
- Tabla 'logs': (id, usuario_id, accion, tabla_afectada, fecha)

6.2 - Relaciones Principales

- usuarios - profesores (1:1)
- usuarios - alumnos (1:1)
- alumnos - notas (1:N)
- alumnos - asistencia (1:N)
- alumnos - equipos (N:M)
- evaluaciones - notas (1:N)
- cursos - profesores (1:N)
- cursos - alumnos (1:N)
- cursos - evaluaciones (1:N)

7. PRÓXIMOS PASOS

Cronograma de Entregas(posibles modificaciones futuras):

- 1) 13/5 (HOY) - Entrega: Hipótesis, Alcance, Backlog, Mockups
- 2) 20/5 - Entrega: Repositorio GitHub, Listado endpoints, Tablas SQL
- 3) 27/5 - Entrega: Templates desarrollados
- 4) 3/6 - Entrega: Integración Front-End con Backend
- 5) 8/6 - Entrega: Primera versión funcional
- 6) 17/6 - 1era defensa del proyecto
- 7) 22/6 - Defensa y consultas
- 8) 24/6 - 2da entrega final

Actividades Inmediatas:

Crear repositorio en GitHub con estructura base

Configurar GitHub Projects con este Kanban

Asignar roles dentro del equipo (Backend, Frontend, BD)

Crear ramas por funcionalidad (feature branches)

Iniciar Sprint 1: Infraestructura